

An empirical study of the impact of software design patterns on code quality

The article, "**An empirical study of the impact of software design patterns on code quality**," investigates the relationship between the application of design patterns and various aspects of software quality. Using a quantitative approach, the researchers gathered data from experienced software developers to evaluate how these patterns affect code in real-world scenarios.

Core Findings

The study concludes that the use of software design patterns is positively associated with overall code quality. Key benefits identified include:

- **Reduced Complexity:** The application of design patterns helps manage and reduce the inherent complexity of software code.
- **Improved Maintainability:** Patterns contribute to higher levels of maintainability, making it easier for developers to update or fix the system over time.
- **Standardized Solutions:** They provide standardized solutions to common design problems, which facilitates a shared understanding among development teams.

Factors Influencing Success

The research emphasizes that the benefits of design patterns are not automatic but depend on certain variables:

- **Developer Experience:** The study highlights that a developer's level of experience with software design patterns is a crucial factor in successfully improving code quality.
- **Appropriate Application:** While generally beneficial, the study suggests that the positive impact is most realized when patterns are applied correctly to solve specific, recurring design issues.

Significance to Software Engineering

The findings suggest that software development teams should actively consider the use of design patterns as a strategic tool to enhance code quality. By providing a quantitative basis for their benefits, the study encourages the integration of pattern-based design into standard software engineering practices to ensure more robust and maintainable systems.

Empirical Research on Pattern Impact

While the notes mention benefits like "reusability" and "maintainability", academic research often investigates if these claims hold true in large-scale systems.

Suggested Reading:

1. An empirical study of the impact of software design patterns on code quality
2. Impact of Design Patterns on Software Maintainability
3. Impact of design patterns on software quality: a systematic literature review

Research in this area often uses software metrics to prove whether patterns like the Strategy, Decorator or Observer actually reduce "code smells" or if they lead to "overuse" and unnecessary complexity, a drawback noted in your week 3 materials.

This article, titled "Impact of Design Patterns on Software Maintainability" (2014), explores the controversial relationship between the use of Gang of Four (GoF) design patterns and the actual ease of maintaining software systems.

Core Objective

The study aims to provide a more objective way to determine which design patterns improve maintainability and under what specific conditions. The authors note that while patterns are intended to improve quality, empirical results in the field remain inconsistent, with some studies suggesting that patterns can actually make systems less maintainable due to increased complexity.

Key Research Findings

The article highlights several critical factors that influence whether a design pattern is beneficial:

- **Trade-offs:** Applying a pattern is rarely "universally good"; it is usually a trade-off where benefits like reusability are weighed against the complexity added to the system.
- **Influencing Factors:** The effect of a pattern on maintainability is heavily influenced by the **system size**, the **prior expertise of the developer**, and the specific **quality attributes** prioritized by the team.
- **Pattern Selection:** The study suggests that patterns are most effective in systems intended for heavy reuse or long-term maintenance.

What We Can Learn From Both Articles

Synthesizing the findings from these research papers reveals several key lessons for software architects and developers:

- **Patterns are Not "Magic Bullets":** Both articles suggest that patterns are not universally good or bad. Their effectiveness depends on the specific context of the project, such as the system size and the long-term maintenance goals.
- **The "Human Factor" is Critical:** A recurring theme in both papers is that the **experience and expertise** of the developer is a primary factor in whether a design pattern actually improves the code. Inexperienced developers may misuse patterns, potentially leading to increased complexity or "code smells".
- **Decision-Making requires Metrics:** To avoid the "controversial" results mentioned by Alghamdi and Qureshi, developers should use objective **maintainability predictors**—such as Depth of Inheritance Tree (DIT), Number of Children (NOC), and Message-Passing Coupling (MPC)—to evaluate the impact of a pattern before full implementation.
- **Complexity Management:** While patterns introduce more classes and abstractions (which can seem complex), their primary value lies in reducing the *overall* complexity of the system logic, making it easier for teams to fix and extend the software over time.
- **The Importance of "Intent":** Choosing a pattern should be guided by the "intent" and the specific axes of change (e.g., whether you need to add new concrete implementers or new clients), rather than just following a blueprint.

Impact of design patterns on software quality: a systematic literature review

The article "**Impact of design patterns on software quality: a systematic literature review**" (2020) by Fadi Wedyan and Somia Abufakher provides a comprehensive analysis of how Gang of Four (GoF) design patterns influence software quality attributes, aiming to resolve contradictory findings in previous research.

Core Objective

The primary goal of this study is to explain the inconsistent results of existing evaluations by identifying the confounding factors, metrics, and implementation practices that affect software quality when design patterns are used. The authors also seek to provide recommendations for improving the structure and implementation of patterns to better support high-quality software development.

Key Findings

The review highlights several critical factors that determine the impact of design patterns on quality:

- **Documentation is Crucial:** Evidence shows that documenting design pattern instances (e.g., using source code comments) significantly improves program comprehension and reduces maintenance costs in terms of time and errors.
- **Modularity and AOP:** The study identifies Aspect-Oriented Programming (AOP) as a promising approach to modularize design patterns, which can help reduce code tangling and improve the separation of concerns.
- **Impact of Size and Scattering:** The size of pattern-related classes and the degree of pattern scattering are identified as major factors influencing overall software quality.
- **Lack of Consensus:** The authors found no firm agreement on whether classes participating in design patterns are more fault-prone or change-prone than non-participating classes.
- **Metric Inconsistency:** A significant challenge in this field is the use of non-unified metrics and different "modules" for measurement across studies, which makes it difficult to compare results.

The third article, the **Systematic Literature Review (SLR)** by Wedyan and Abufakher (2020), is arguably the most important "research-based" addition because it doesn't just look at one experiment; it synthesizes **18 years of research (50 primary studies)**.

Here is what we can learn from this "big picture" view that wasn't as clear in the first two articles:

1. Implementation Style is as Important as the Pattern Itself

This study introduces a more modern research angle: **Aspect-Oriented Programming (AOP)**.

- It suggests that traditional Object-Oriented (OO) implementations of patterns (like those in your lecture notes) sometimes lead to "code tangling."
- **The Lesson:** Research is moving toward using "Aspects" to separate pattern logic from business logic, which can result in much cleaner code than the standard GoF implementation.

2. Documentation is a "Force Multiplier"

A unique takeaway from this SLR is the empirical proof that **documenting the pattern** (e.g., a simple comment saying `// This is a Strategy Pattern implementation`) significantly lowers maintenance time.

- Even if the code is technically "perfect," if the next developer doesn't recognize the pattern, the quality benefits are lost.
- **The Lesson:** Professional software engineering requires explicit communication of design intent, not just clean code.

3. There is No "Standard" Measurement for Quality

The SLR reveals a major weakness in software engineering research: different researchers use different metrics (e.g., some use "bugs found," others use "time to change code").

- **The Lesson:** When you read that a pattern "improves quality," you must ask: *"How exactly was quality measured in this context?"*

What is AOP? How to implement Design Patterns using AOP?

Aspect-Oriented Programming (AOP) is a programming paradigm that aims to increase modularity by allowing the separation of **cross-cutting concerns**.

In traditional Object-Oriented Programming (OOP), certain tasks—like logging, security, or transaction management—often end up scattered across many different classes. This leads to "code tangling" (where a single method does too many things) and "code scattering" (where the same code is repeated in many places). AOP solves this by allowing you to define these behaviors in a single place (an **Aspect**) and "weave" them into the code where needed.

Key AOP Concepts

- **Aspect:** A module that encapsulates a cross-cutting concern (e.g., a Logging Aspect).
- **Join Point:** A specific point in the execution of the program, such as a method call or an exception being thrown.
- **Advice:** The action taken by an aspect at a particular join point (e.g., "log this message *before* the method runs").
- **Pointcut:** A regular expression or logical set of rules that matches join points to determine where advice should be applied.

Implementing Design Patterns using AOP

Research, such as the systematic literature review by Wedyan and Abufakher (2020), suggests that implementing design patterns with AOP can lead to better modularity than traditional OOP. In OOP, the "pattern code" is often mixed with "business code." With AOP, you can extract the pattern's logic into an Aspect.

Example: The Strategy Pattern

In OOP, a "Context" class must hold a reference to a Strategy interface and call its methods.

- **AOP Implementation:** AOP can be used to dynamically switch strategies based on the execution context (like a user's permission level or system load) without the Context class ever having to explicitly manage the strategy object.

Why use AOP for Patterns?

1. **Cleaner Business Logic:** The classes in your system focus purely on their primary responsibility, while the "Design Pattern" logic is moved to the Aspect.
2. **Pluggability:** You can add or remove a design pattern (like switching from a Singleton to a Prototype) by simply changing the Aspect configuration, without modifying the core source code.
3. **Better Reusability:** Aspects can often be applied to multiple different classes across a project, whereas an OOP pattern must be manually implemented in every relevant class.

To illustrate how Aspect-Oriented Programming (AOP) transforms a design pattern, let's look at a common scenario: a **Payment Processor** that needs to switch between different calculation strategies (e.g., Normal, Discount, or VIP) based on a user's status.

1. The Traditional OOP Approach

In standard Object-Oriented Programming, the "Context" (the class using the strategy) must be aware of the strategy interface and often contains logic to set or switch the strategy.

```
// Context class
public class OrderProcessor {
    private PaymentStrategy strategy; // The Strategy interface

    public void setStrategy(PaymentStrategy strategy) {
        this.strategy = strategy;
    }

    public void process(double amount) {
        // Business logic is mixed with strategy invocation
        double finalAmount = strategy.calculate(amount);
        System.out.println("Processing payment: " + finalAmount);
    }
}
```

The Problem: The `OrderProcessor` is "polluted" with code related to managing the strategy. If you want to add a new rule for when a strategy is applied, you usually have to modify the `OrderProcessor` or the client calling it.

2. The AOP Approach

In AOP, we keep the `OrderProcessor` completely "clean." It doesn't even know a Strategy pattern exists. We use an Aspect to intercept the method call and inject the correct behavior.

```
public class OrderProcessor {
    public void process(double amount) {
        // Pure business logic only
        System.out.println("Processing payment: " + amount);
    }
}
```

The Context

```
@Aspect
public class PaymentStrategyAspect {

    // 1. Define where to "weave" the logic (Pointcut)
    @Pointcut("execution(* OrderProcessor.process(..) && args(amount)")
    public void paymentExecution(double amount) {}

    // 2. Define the logic to inject (Advice)
    @Around("paymentExecution(amount)")
    public Object applyStrategy(ProceedingJoinPoint joinPoint, double amount) throws Throwable {
        PaymentStrategy strategy;

        // Strategy selection logic is centralized here, not in the business class
        if (UserContext.isVIP()) {
            strategy = new VipDiscountStrategy();
        } else {
            strategy = new NormalStrategy();
        }

        double finalAmount = strategy.calculate(amount);

        // Execute the original method with the NEW calculated amount
        return joinPoint.proceed(new Object[] { finalAmount });
    }
}
```

The Aspect

Why AOP is "Better" in this Context

Based on the research from the Systematic Literature Review (Wedyan & Abufakher, 2020) and the study on Maintainability (Alghamdi & Qureshi, 2014), here is why the AOP implementation is often superior for complex systems:

- **Decoupling (Separation of Concerns):** In OOP, the `OrderProcessor` is coupled to the `PaymentStrategy` interface. In AOP, the `OrderProcessor` is 100% decoupled. You could remove the entire Strategy pattern by simply deleting the Aspect, and the core code remains functional.
- **Elimination of "Code Tangling":** Your lecture notes mention that behavioral patterns deal with the "assignment of responsibilities." AOP takes this further by moving the responsibility of *choosing* and *executing* the strategy into a separate module (the Aspect), leaving the business logic "pure."
- **Centralized Policy Management:** If your "VIP Discount" logic changes or you add a "Holiday Sale" strategy, you only change the Aspect. In OOP, you might have to find every place `setStrategy()` is called across the entire codebase.
- **Reduced Scattering:** In large systems, the same strategy might need to be applied to many different methods (e.g., `calculateTax`, `calculateShipping`). In OOP, you'd add strategy code to all those methods. In AOP, you just update the **Pointcut** to include those methods.

Finally, a Note of Caution from Research:

As noted in the Wedyan review, AOP can make debugging harder because the behavior happens "behind the scenes." For this reason, documentation (naming your Aspects and Pointcuts clearly) becomes the most critical factor for maintainability.